



Universidad Nacional Autónoma de México

Facultad de Estudios Superiores plantel Aragón



Temas especiales de computación 1



Sistema de validación documental con código QR

Integrantes: Luis Angel Castañeda Castañon

Profesor: Aaron Velasco Agustin

Grupo: ED11

Fecha de entrega: 24 de Junio de 2026

Índice

Descripción del programa.....	3
Tecnologías usadas.....	4
Descripción general.....	6
Herramientas de análisis.....	7
Casos de uso.....	7
Caso de Uso 1: Crear Figura.....	7
Caso de Uso 2: Aplicar Transformación.....	8
Caso de Uso 3: Consultar Historial de Transformaciones.....	9
Diagrama de Gantt.....	9
Componentes / especificaciones generales.....	11
ÁTOMOS.....	11
Canvas.....	11
MOLECULE.....	14
ShapePoints.....	14
ORGANISM.....	16
ShapeState.....	16
ShapeFunctionState.....	18
page.tsx.....	20
Patrones de diseño.....	22
Strategy.....	22
Conclusiones.....	23

Descripción del programa

El Sistema es una aplicación web diseñada para garantizar la validez de los documentos PDF que se registran en la aplicación.

El sistema permite que usuarios autorizados inicien sesión y carguen archivos PDF, capturando la información importante del documento, algunos datos que se ocupan son: título, área que emite el documento y cuál es su tipo de documento. Durante el proceso, la plataforma genera un folio único y un código QR asociado al documento, el cual es insertado automáticamente dentro del archivo PDF en la zona que se solicite.

Una vez registrado, el documento queda almacenado en una carpeta temporal dentro del sistema (la carpeta es temporal tratando de simular una carga de un archivo en algún proveedor de nube, por ejemplo S3, para solo mantener la url del archivo en la base de datos). Los usuarios autorizados pueden consultar los documentos registrados, descargar la versión con QR y administrar su estado como revocación o cancelación.

El código QR generado contiene una liga de validación pública que puede ser consultada por cualquier persona sin necesidad de iniciar sesión. Al escanear el código o acceder a la liga de validación, el sistema muestra el estado actual del documento, indicando si se encuentra vigente, revocado, cancelado o si no existe registro del mismo.

Tecnologías usadas

Componente	Tecnología Seleccionada	Justificación Técnica PDF
Backend Framework	NestJS (Node.js)	Proporciona una arquitectura mantenible basada en módulos e inyección de dependencias. Facilita la separación de responsabilidades y la integración nativa con TypeScript. A parte de que por gusto personal, recientemente complete un curso sobre NestJS y me pareció un gran momento para ponerlo en practica
Frontend Framework	Next.js (React)	Permite un enrutamiento ágil gracias a App Router, un manejo de vistas protegidas para los usuarios internos y vistas públicas para la validación.
Base de Datos	PostgreSQL	Esta es la base de datos que mejor se acomodaba con el ORM que en la documentación de NestJS recomendaban.
ORM	Prisma	Permite un mapeo objeto-relacional con tipado estático automático a partir del esquema de la base de datos, además de gestionar las migraciones del sistema de forma versionada.
Orquestación	Docker Compose	Permite empaquetar y levantar todo el ecosistema (frontend, backend y base de datos) con un único comando, de igual manera ejecutamos un comando para eliminar las cosas innecesarias y borrar los volúmenes temporales.
Autenticación	JWT + Cookies	Implementa un esquema de seguridad donde el token de sesión se almacena del lado del cliente sin ser accesible por JavaScript, mitigando ataques de tipo XSS.

Manipulación de PDF y QR	qrcode + pdf-lib	Permiten la generación dinámica de la imagen del código QR en el servidor y su posterior inyección asíncrona en las coordenadas especificadas del documento PDF.
Gestión de Archivos	Multer	Middleware encargado de procesar las peticiones HTTP de tipo multipart/form-data para recibir y almacenar los archivos PDF subidos por los usuarios.

Flujo del sistema

A continuación, se detalla el flujo lógico paso a paso que realiza la plataforma, dividiéndose en el proceso administrativo (interno) y el proceso de validación (público):

A. Proceso de Carga y Registro (Usuario Interno)

1. Autenticación: El usuario (con rol de ADMIN o CAPTURISTA) ingresa sus credenciales en el módulo de login. El servidor valida los datos y retorna un JWT con los datos del usuario: rol, nombre, email, id; firmado dentro de una cookie estructurada httpOnly.
2. Acceso al Panel: El usuario es redirigido al panel principal (En este se encuentran los botones de navegación, por el momento solo es el de documentos).
3. Carga de Documento: El usuario selecciona el botón de agregar documento, accede al formulario de carga, selecciona un archivo PDF local y captura los datos obligatorios: Título, Tipo de documento, Área emisora y la Posición deseada para el código QR.
4. Procesamiento en Backend:
 - El sistema genera un folio único para el documento (UUID).
 - Se genera un código QR que apunta hacia la URL pública de validación (/validate/:folio).
 - La librería pdf-lib lee el archivo original e inserta la imagen del QR en la posición seleccionada.
5. Almacenamiento: El sistema guarda de forma local (simulando un almacenamiento en la nube) tres recursos: el PDF original, el PDF modificado con el QR y la imagen aislada del QR. Adicionalmente, Prisma inserta los registros en la base de datos con el estado inicial VIGENTE y asocia el ID del usuario creador.

B. Proceso de Consulta y Gestión (Usuario Interno)

1. Los usuarios autorizados pueden examinar la tabla con el listado de documentos, descargar las versiones con el código QR incrustado o copiar de manera directa la liga de validación.
2. Modificación de Estado: Si un documento pierde validez, un usuario con el rol ADMIN puede cambiar su estado a REVOCADO o CANCELADO desde la vista de detalle. Esta acción registra de forma automática un evento en la tabla de Bitácora.

C. Proceso de Verificación (Usuario Público)

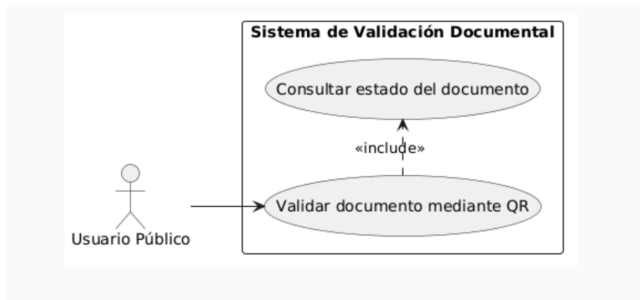
1. Escaneo del QR / Acceso al Enlace: Cualquier usuario puede escanear el código QR impreso o digital en el PDF, o ingresar directamente a la URL pública de validación sin necesidad de autenticarse.
2. Despliegue del Estado: El frontend realiza una petición al backend consultando el folio. La plataforma responde y despliega una interfaz de consulta con alguno de los siguientes escenarios:
 - Válido / Vigente: Muestra la información del documento confirmando su autenticidad.
 - Revocado / Cancelado: Informa explícitamente que el documento ya no cuenta con validez oficial.
 - No encontrado: Se notifica si el folio consultado no existe en los registros del sistema.
3. Registro de Auditoría: Cada consulta realizada a través de la interfaz pública inserta un registro en la tabla DocumentValidation para almacenar un histórico de las verificaciones del documento.

Diagrama de caso de uso

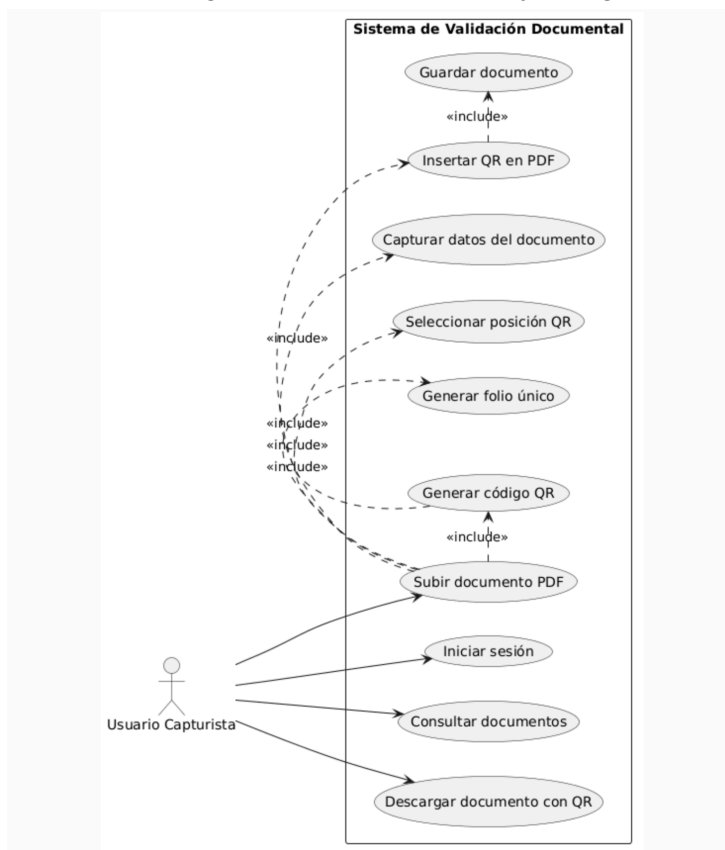
Actores del sistema

Aquí se muestra una breve descripción de quienes son los usuarios

1. Usuario Público: Cualquier persona externa a la organización que interactúa con la plataforma a través de un dispositivo móvil o navegador web al escanear el código QR de un documento físico o digital. No requiere autenticación.



2. Usuario Capturista: Personal interno autorizado encargado de alimentar el sistema mediante la carga de documentos PDF y el registro de sus metadatos iniciales.



3. Administrador: Usuario con privilegios elevados dentro del sistema. Además de poder realizar las funciones del Capturista, cuenta con la facultad exclusiva de revocar o cancelar la validez de los documentos oficiales y consultar quién ha cargado los documentos o actualizado su estado.



Descripción de los Casos de Uso Principales

Caso de Uso: Iniciar sesión

- Actor: Usuario Capturista / Administrador
- Descripción: Permite al usuario del sistema ingresar sus credenciales para obtener un token de acceso seguro (JWT) que lo habilite a interactuar con los módulos internos protegidos.

Caso de Uso: Subir documento PDF

- Actor: Usuario Capturista / Administrador
- Descripción: Es la acción principal de registro. El usuario selecciona el archivo original y completa el formulario con el título, tipo de documento y área emisora. Este caso de uso incluye de manera obligatoria la generación del folio único, la creación del código QR, la selección de la posición y la inserción del QR sobre el archivo final antes de guardarlo de forma local en el servidor.

Caso de Uso: Consultar documentos

- Actor: Usuario Capturista / Administrador
- Descripción: Acceso al repositorio general donde se listan en una tabla todos los registros de los documentos guardados, visualizando sus folios, metadatos y estados vigentes.

Caso de Uso: Descargar documento con QR

- Actor: Usuario Capturista / Administrador
- Descripción: Permite descargar directamente el archivo PDF procesado que ya contiene incrustado el código QR en las coordenadas predefinidas para su distribución oficial.

Caso de Uso: Revocar / Cancelar documento

- Actor: Administrador
- Descripción: Permite cambiar de forma definitiva el estado de un documento a Revocado o Cancelado. Esta acción actualiza de inmediato el resultado de la validación pública y genera un historial en la bitácora del sistema.

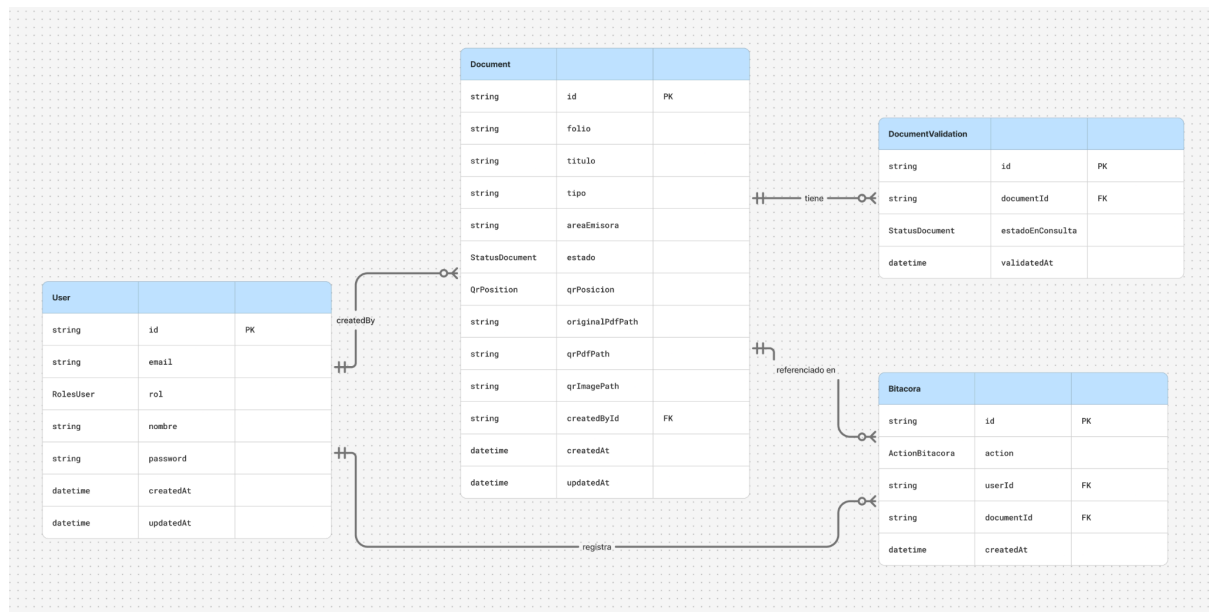
Caso de Uso: Validar documento mediante QR

- Actor: Usuario Público / Administrador / Usuario Capturista
- Descripción: El usuario escanea el QR. El sistema procesa la petición del folio y despliega una interfaz indicando si el archivo es vigente, revocado, cancelado o si no existe en los registros.

Diagrama entidad - relación

En esta sección se detalla la estructura lógica de la base de datos relacional implementada en PostgreSQL a través del ORM Prisma. El diseño consta de cuatro entidades principales, tipos enumerados para asegurar la integridad de los datos y sus respectivas relaciones cardinales.

En esta sección no aparece la tabla de estados, porque considero que podía manejarlo correctamente con un Enum y así no hacer una tabla extra



ENUMS

Estos tipos de dato nos permiten tener definidos los valores únicos que son permitidos en nuestras filas de la base de datos

1. RolesUser: Define los roles del sistema.
2. StatusDocument: Define los estados permitidos: VIGENTE, REVOCADO y CANCELADO.
3. QrPosition: Los valores permitidos son SUPERIOR_DERECHA, SUPERIOR_IZQUIERDA, INFERIOR_DERECHA, INFERIOR_IZQUIERDA y ULTIMA_PAGINA_INFERIOR_DERECHA.
4. ActionBitacora: Los valores permitidos son CREATE, REVOKE y CANCEL. Estos son las acciones que se guardan en la bitacora

Descripción de las Entidades y Atributos

User: Representa a los usuarios autenticados que gestionan la plataforma.

- id (String, PK): Identificador único de usuario generado mediante formato UUID.
- email (String): Correo electrónico institucional utilizado como credencial de acceso (único).
- rol (RolesUser): Tipo de rol asignado, por defecto CAPTURISTA.
- nombre (String): Nombre completo del usuario administrativo o capturista.

- password (String): Contraseña encriptada para el control de accesos.
- createdAt (DateTime): Fecha y hora exacta de la creación del usuario.
- updatedAt (DateTime): Fecha y hora del último cambio en el registro del usuario.

Document: Almacena los datos de los documentos registrados y las referencias físicas de los archivos en el servidor.

- id (String, PK): Identificador único del documento en formato UUID.
- folio (String): Identificador único de validación pública asignado por el sistema.
- titulo (String): Nombre del documento.
- tipo (String): Tipo de documento cargado.
- areaEmisora (String): Área responsable de la expedición del archivo.
- estado (StatusDocument): Estado de vigencia actual del documento, por defecto VIGENTE.
- qrPosicion (QrPosition): Posición elegida para la incrustación del código.
- originalPdfPath (String): Ruta de almacenamiento local de la copia del archivo PDF original.
- qrPdfPath (String): Ruta de almacenamiento de la versión final del PDF que contiene el QR incrustado.
- qrImagePath (String): Ruta de la imagen aislada del código QR generado.
- createdById (String, FK): Llave foránea que vincula al usuario que realizó el registro.
- createdAt (DateTime): Fecha y hora en la que se dio de alta el documento.
- updatedAt (DateTime): Fecha y hora de la última actualización (cambio de estado).
-

DocumentValidation: Registra el historial cada vez que se consulta la URL pública.

- id (String, PK): Identificador único de la consulta en formato UUID.
- documentId (String, FK): Llave foránea que asocia la validación al documento consultado.
- estadoEnConsulta (StatusDocument): Almacena el estado que presentaba el documento al momento preciso en que se realizó la consulta.
- validatedAt (DateTime): Sello de tiempo automático de la consulta pública.

Bitacora: Permite auditar de forma estricta qué acciones ejecutaron los administradores o capturistas sobre los documentos.

- id (String, PK): Identificador único del movimiento en formato UUID.
- action (ActionBitacora): Tipo de evento registrado (creación, revocación o cancelación).
- userId (String, FK): Llave foránea que identifica al usuario responsable del movimiento.
- documentId (String, FK): Llave foránea que asocia la acción al documento afectado.
- createdAt (DateTime): Fecha y hora exacta en que ocurrió el movimiento administrativo.

Instalación

Requisitos previos: tener Docker y Docker Compose instalados.

Pasos:

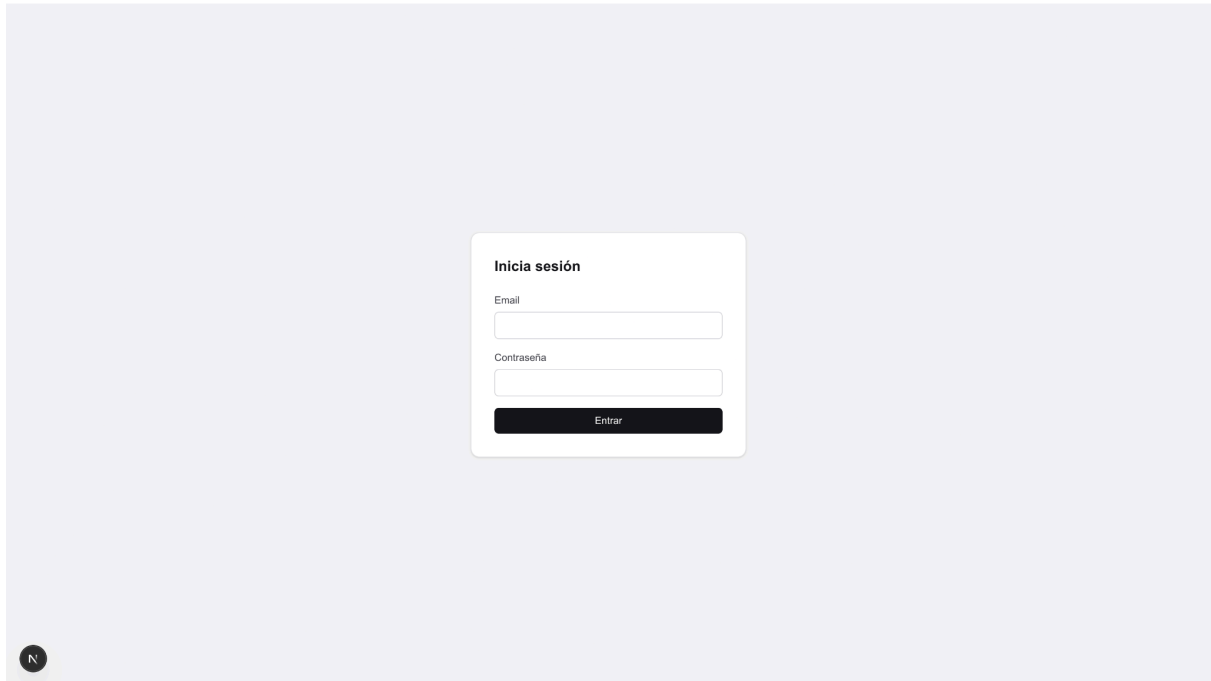
1. Clonar el repositorio:
git clone
https://github.com/AngelCstd/proyecto_final_sistema_validacion_documental.git
2. Entrar a la carpeta del proyecto:
cd proyecto_final_sistema_validacion_documental
3. Ejecutar:
docker compose up -d --build
4. Esperar a que los tres contenedores (postgres, backend, frontend) terminen de levantarse. Al iniciar, el backend automáticamente genera el cliente de Prisma, aplica las migraciones (crea las tablas) y crea los usuarios de prueba — no se requiere ningún paso manual adicional.
5. Acceder al sistema desde el navegador:
 - a. Frontend: <http://localhost:3001>
 - b. Backend (API): <http://localhost:3000>
6. Iniciar sesión con cualquiera de los siguientes usuarios de prueba:

Rol	Email	Contraseña
Administrador	admin@prueba.com	admin
Capturista	capturista@prueba.com	capturista

7. Para apagar el sistema:
docker compose down -v

Evidencias del sistema

1. Login



A screenshot of a login page with a light gray background. In the center is a white rounded rectangle containing the login form. The form has the title 'Inicia sesión' in bold. Below it are two input fields: 'Email' and 'Contraseña'. At the bottom of the form is a dark gray button labeled 'Entrar'. In the bottom-left corner of the page, there is a small dark gray circle with a white 'N' inside.

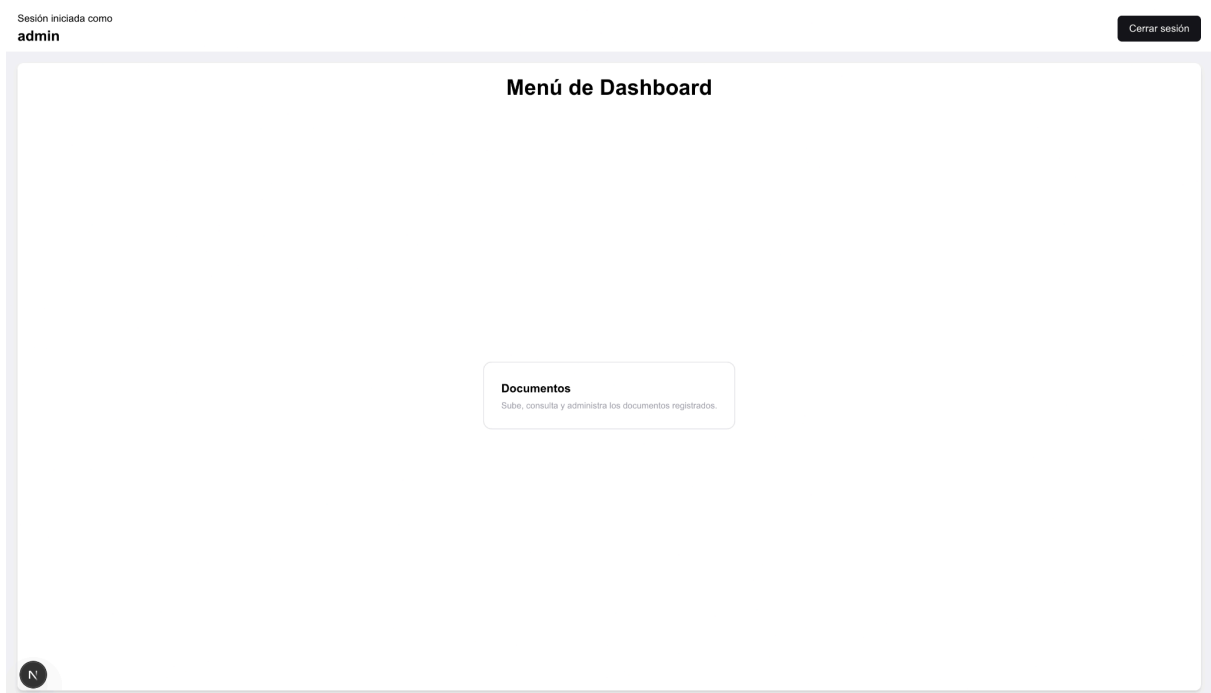
Inicia sesión

Email

Contraseña

Entrar

2. Panel principal



A screenshot of a main dashboard page. At the top left, it says 'Sesión iniciada como admin'. At the top right is a dark gray button labeled 'Cerrar sesión'. The main content area is titled 'Menú de Dashboard' and is mostly empty. In the center, there is a white rounded rectangle with the title 'Documentos' and the text 'Sube, consulta y administra los documentos registrados.' below it. In the bottom-left corner of the page, there is a small dark gray circle with a white 'N' inside.

Sesión iniciada como
admin

Cerrar sesión

Menú de Dashboard

Documentos
Sube, consulta y administra los documentos registrados.

3. Carga de documento

Subir documento

Título

Tipo de documento

Área emisora

Posición del QR

Inferior derecha

Archivo PDF

Seleccionar archivo Sin archivo...elegcionados

Subir documento

4. Documento con QR

F_7419ea4c-1ff6-4d95-912a-df522341bf0b.pdf

1 / 2

80%

Luis Angel Castañeda Castañon

Mexico City | angel.castañeda.castanon@gmail.com | +52 55 5544 1167 |
linkedin.com/in/luis-angel-castaneda-castanon | github.com/AngelCand | portfolio-angelcanda.projects.vercel.app

Perfil Profesional

Desarrollador Frontend con 2 años de experiencia académica y práctica desarrollando aplicaciones web con React y Next.js. Enfocado en la construcción de interfaces modernas, reutilizables y orientadas a la experiencia de usuario. Experiencia en consumo de APIs REST, manejo de estado con hooks, integración de formularios y servicios externos, así como trabajo freelance en proyectos reales para eventos y plataformas web. Acostumbrado a trabajar de forma autónoma y colaborativa, cuidando calidad visual, rendimiento y estabilidad del frontend con experiencia en despliegue de aplicaciones Next.js en Vercel y trabajo en entornos de producción.

Habilidades Técnicas

Frontend: React, Next.js, TypeScript, JavaScript, HTML5, CSS3, Tailwind CSS, SASS, componentes reutilizables, manejo de estado, hooks, debuggencia, consumo de APIs REST.
SSR / SSG: Rendimiento del lado del servidor y generación estática con Next.js.
Testing: Jest, pruebas unitarias básicas en frontend.
Backend para integración: Node.js, Express, REST APIs, validación de datos.
Bases de Datos: MySQL, PostgreSQL, modelado relacional, consultas SQL.
Herramientas: Git, GitHub, Postman, Figma, Vercel.

Experiencia laboral

Frontend Developer (React, Next.js), Nikros - Mexico City Sep 2024 - Actualidad

- Publicación de aplicaciones frontend desarrolladas en React y Next.js mediante Vercel, asegurando disponibilidad y despliegues rápidos así como entornos de prueba.
- Desarrollo de interfaces frontend con React y TypeScript, priorizando mantenibilidad, reutilización de componentes y código limpio cumpliendo con los estándares.
- Implementación de SSR para mejora de carga inicial, experiencia de usuario y SEO mejorando hasta un 30% la carga inicial.
- Consumo e integración de servicios de terceros mediante Node.js para comunicación frontend-backend.
- Coordinación técnica y liderazgo parcial del desarrollo en proyectos full stack.
- Desarrollo de una plataforma de atención automatizada con OpenAI, reduciendo tiempos de respuesta en 95%.
- Migración completa a Next.js, unificando frontend y backend y reduciendo costos de servidor en 50%.
- Integración de servicios externos: OpenAI, Stripe, Icturema, Zoho Desk, Google Maps y Google Sheets.
- Disfraz de más de 20 componentes reutilizables con React, Tailwind y TypeScript, reduciendo tiempos de desarrollo en 60%.

Desarrollador Frontend Freelance 2023 - 2024

- Desarrollo de sitios web enfocados a eventos, incluyendo Expo Emprende Alondra 2024.
- Construcción de interfaces frontend con React, utilizando componentes reutilizables y enfoque en experiencia de usuario.
- Implementación de formularios dinámicos conectados a bases de datos para registro y contacto de usuarios.
- Integración inicial de automatización de correos con Zapier; posterior migración a SendGrid para mejorar estabilidad y control del envío.
- Consumo de APIs REST y manejo de datos en formato JSON.
- Uso de Git para control de versiones y organización del desarrollo.

Educación

Ingeniería en computación, Hex Aragón, UNAM - Mexico City Ago 2023

Formación enfocada en desarrollo de software, lógica de programación y sistemas. Desarrollo de aplicac

5. Pagina publica de validación

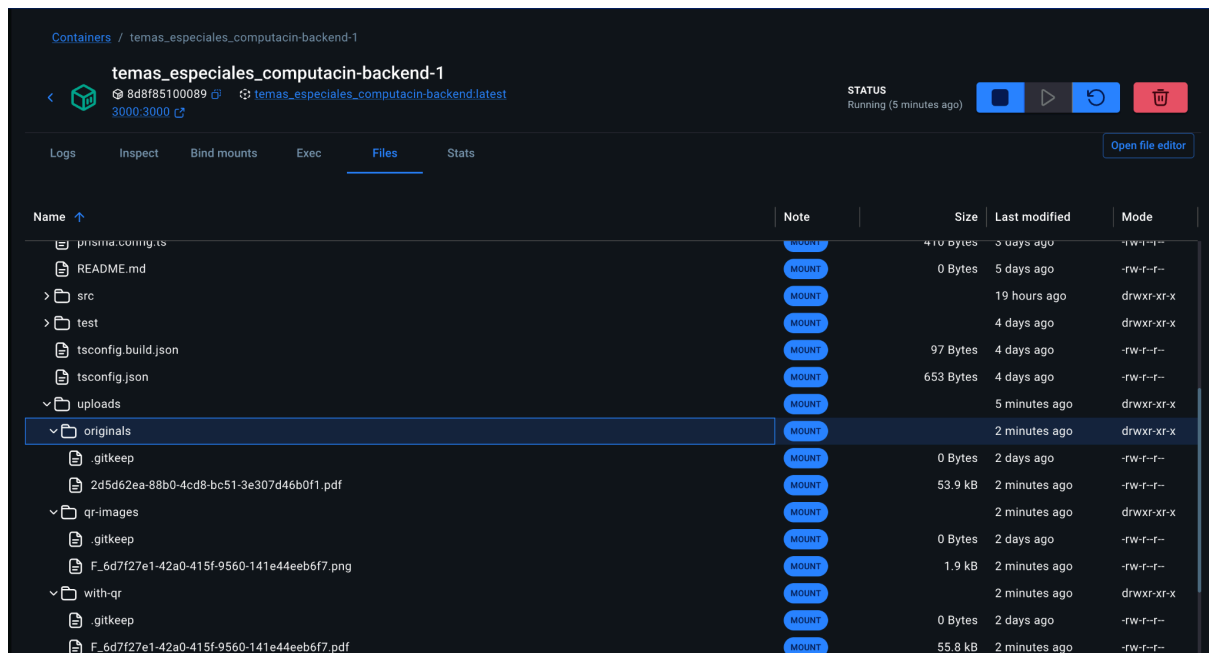
Curriculum Angel Castañeda

Folio
F_6d7f27e1-42a8-415f-9560-141e44eeb6f7

Vigente

6. Repositorio de documentos

En esta pantalla podemos ver en la parte de uploads las 3 carpetas donde guardamos el original, el documento con el pdf y la imagen sola del qr



7. Revocación o cambio de estado

Curriculum Angel Castañeda Vigente

Información

Validaciones

Bitácora

Folio

F_6d7f27e1-42a0-415f-9560-141e44eeb6f7

Tipo

CV

Área emisora

Recursos Humanos

Posición del QR

Inferior derecha

Fecha de registro

24/6/2026, 6:23:22 p.m.

Descargar

Acciones

Revocar

Cancelar

Curriculum Angel Castañeda

Folio

F_6d7f27e1-42a0-415f-9560-141e44eeb6f7

Revocado